Implement & Manage CIS Benchmarks for macOS Devices for Microsoft Intune

Implementation Solution Guide

Version 1.0.0 · 2026-06-29

Contents

Executive Summary — CIS Benchmarks for macOS (Microsoft Intune) 5
- What you get . . . 5
- What this delivers . . . 5
- The deployment model . . . 5
- Outcome . . . 6

1. Solution Overview — CIS Benchmarks for macOS (Microsoft Intune) 6
- What the CIS Benchmarks are . . . 6
- Level 1 vs Level 2 . . . 6
- How the controls map into Intune (the three macOS surfaces) . . . 6
- CIS Benchmark vs the Microsoft / Apple defaults . . . 7
- The compliance and drift model . . . 7
- The benchmark-version maintenance lifecycle (why this solution is two solutions in one) . . . 7
- Source-of-truth note (CIS membership content) . . . 7
- Benefits . . . 7
- Limitations (be honest with buyers) . . . 8
- The deployment model . . . 8

2. Requirements Guide — CIS Benchmarks for macOS (Microsoft Intune) 8
- 2.1 Intune licensing . . . 9
- 2.2 Apple MDM Push (APNs) certificate . . . 9
- 2.3 macOS enrollment configured . . . 9
- 2.4 Supported macOS version . . . 9
- 2.5 Official CIS Benchmark source (membership content) . . . 9
- 2.6 Custom `.mobileconfig` payloads for non-catalog controls . . . 10
- 2.7 Pilot and production device groups . . . 10
- 2.8 Change control / approval process . . . 10
- 2.9 (Optional) Application inventory for break-glass exceptions . . . 10
- Checklist . . . 10

Global Requirements — Intune Solutions (category-level) 11
- G1. Microsoft Entra ID tenant . . . 11
- G2. Microsoft Intune subscription . . . 11
- G3. Administrative roles . . . 11
- G4. Device enrollment in Intune . . . 11
- G5. Pilot security group . . . 11
- G6. Supported Windows baseline . . . 12
- G7. Network access to Microsoft cloud endpoints . . . 12

3. Implementation Guide (net-new) — CIS Benchmarks for macOS (Microsoft Intune) 12
Step 1 — Stage the benchmark (Assess) . . . 12
Step 2 — Confirm Apple management prerequisites . . . 13
Step 3 — Create the device groups . . . 13
Step 4 — Build the Level 1 settings-catalog policy . . . 13
Step 5 — (Optional) Build the Level 2 settings-catalog policy . . . 13
Step 6 — Add custom `.mobileconfig` profiles (only if needed) . . . 13
Step 7 — Add macOS shell scripts (only for script-only controls) . . . 14
Step 8 — Handle break-glass exceptions . . . 14
Step 9 — Create the compliance policy . . . 14
Step 10 — Apply and validate on the pilot, with pilot gate and rollback note . . . 14
Step 11 — Expand to production . . . 15
4. Maintenance & Update Guide — the CIS benchmark-version lifecycle (macOS) 15
Two kinds of maintenance . . . 15
Part A — Ongoing drift maintenance (run on a cadence) . . . 16
Part B — Benchmark-version update (run when CIS releases a new version) . . . 16
5. Testing Guide — CIS Benchmarks for macOS (Microsoft Intune) 18
Test 1 — Settings-catalog policy applied to the pilot . . . 18
Test 2 — Settings present on the device . . . 18
Test 3 — Compliance policy evaluates the device . . . 18
Test 4 — Custom `.mobileconfig` profile applied (if used) . . . 19
Test 5 — Shell-script control applied (if used) . . . 19
Test 6 — Level 2 controls present (if deployed) . . . 19
Test 7 — App and login soak (hardening did not break the business) . . . 19
Test 8 — Fleet posture and drift report . . . 19
Test 9 — Benchmark-version diff (maintenance dry-run) . . . 19
Pass/fail gate . . . 19
6. Troubleshooting Guide — CIS Benchmarks for macOS (Microsoft Intune) 20
A. Policy/profile not applying / device not compliant . . . 20
B. Hardening broke an app or login . . . 20
C. Custom `.mobileconfig` profile not taking effect . . . 21
D. macOS shell-script control not enforcing . . . 21
E. Compliance policy reports noncompliant unexpectedly . . . 21
F. Benchmark-version update issues . . . 21
G. Graph scripts fail to connect . . . 22
H. Diagnostics quick reference . . . 22

I. When to escalate . . . 22
7. Best Practices — CIS Benchmarks for macOS (Microsoft Intune) 22
8. Deliverables Summary — CIS Benchmarks for macOS (Microsoft Intune) 23
Documentation . . . 23
PowerShell scripts . . . 24
Object naming convention (used across docs and scripts) . . . 24
What the buyer receives . . . 24
Outcome . . . 25
Appendix A — PowerShell Scripts 26

Executive Summary — CIS Benchmarks for macOS (Microsoft Intune)

Security teams, auditors, and cyber-insurers increasingly require Macs to meet the **CIS Benchmarks** — but doing it by hand is brutal: hundreds of settings split across configuration profiles, controls the Intune GUI can't reach (so you need custom `.mobileconfig` payloads and shell scripts), Level 1 vs Level 2 decisions, controls that break apps or even login, and a benchmark that CIS *revises* regularly (and re-publishes for every new macOS release) so your hard-won baseline silently drifts out of compliance. This package implements the CIS Benchmark for macOS as Microsoft Intune policy, proves compliance, and gives you a documented process to keep the baseline current every time CIS publishes a new version.

This is a complete, **deploy-it-today** package written for an administrator with no prior hardening experience — and it is deliberately two solutions in one: **net-new implementation** and an **ongoing update-and-maintain lifecycle**.

What you get

- A plain-language **Solution Overview** of CIS Benchmarks, Level 1 vs Level 2, how controls map to the three Intune macOS surfaces (settings catalog, custom `.mobileconfig` profiles, and shell scripts), and how CIS differs from vendor defaults.
- A **Requirements Guide** with why / verify / implement-if-missing for every prerequisite — including the Apple MDM Push (APNs) certificate and macOS enrollment — plus a consistent object-naming convention.
- An **Implementation Guide** of precise, click-by-click Intune steps to build, pilot, validate, and expand the baseline — with a pilot gate and a rollback note because hardening can break apps and login.
- A **Maintenance & Update Guide** — a first-class lifecycle for every new CIS benchmark version: obtain it, diff it against your live baseline, assess and approve, update in a controlled change, re-validate, and record the version.
- Five **PowerShell scripts** for CIS readiness discovery, per-device compliance validation, fleet posture and drift reporting, safe drift remediation, and benchmark-version comparison.
- **Testing** and **Troubleshooting** guides for confident rollout and ongoing operation.

What this delivers

A **hardened, CIS-compliant Mac fleet** built as Intune settings-catalog policy, custom `.mobileconfig` profiles, and macOS shell scripts at Level 1 (with optional, tested Level 2), provable per-device compliance, drift detection and remediation, and a documented benchmark-version update lifecycle so the baseline never silently falls behind.

The deployment model

Assess → **Configure** → **Pilot** → **Validate** → **Expand** — and, for each new benchmark version, **Obtain** → **Diff** → **Assess/Approve** → **Update** → **Re-validate** → **Record.**

Outcome

Every Mac hardened to the CIS Benchmark and provably compliant — kept current as CIS revises it instead of silently drifting — deployable by a single administrator and validated on a pilot before fleet rollout.

1. Solution Overview — CIS Benchmarks for macOS (Microsoft Intune)

What the CIS Benchmarks are

The CIS Benchmarks are a set of consensus-developed, prescriptive security configuration recommendations published by the Center for Internet Security. The CIS Apple macOS Benchmark is a single document (published per macOS major version, for example a Sonoma or Sequoia benchmark) containing hundreds of individual control recommendations — each one a specific setting, the recommended value, the rationale, and the audit/remediation guidance. Auditors, cyber-insurers, and security teams increasingly treat "meets the CIS Benchmark for macOS" as the bar for a hardened Mac. This solution implements that benchmark as Microsoft Intune policy and then keeps it current as CIS revises the benchmark over time.

Level 1 vs Level 2

- **Level 1 (L1)** — the baseline. Practical, broadly safe hardening that reduces attack surface

with minimal impact on day-to-day use. This is what most Mac fleets deploy first.

- **Level 2 (L2)** — defense-in-depth for high-security environments. Stronger, but more likely to

break apps or workflows (for example stricter Gatekeeper, firewall, or removable-media controls, or disabling features users rely on). Apply L2 selectively, on top of L1, after extra testing.

- The macOS benchmark also covers areas such as FileVault disk encryption, the application firewall,

Gatekeeper, software-update behaviour, and audit logging. Treat the encryption controls as an add-on profile, not a replacement for L1.

How the controls map into Intune (the three macOS surfaces)

macOS is configured by Apple **configuration profiles** (`.mobileconfig` payloads) and by scripts — not by Windows constructs. There is no ADMX and no Windows settings catalog on macOS. The CIS macOS controls translate into Intune through three delivery surfaces:

1. **Settings catalog (Apple)** — the primary surface. Most CIS controls correspond to an Apple

payload Microsoft exposes in the macOS settings catalog (Devices → macOS → Configuration → Create → Settings catalog), grouped by area (for example Login Window, Restrictions, Security, Software Update). You build one or more settings-catalog policies that carry the L1 (and optionally L2) values.

1. **Custom configuration profiles (`.mobileconfig`)** — for the minority of CIS controls whose

Apple payload is not (yet) surfaced in the settings catalog, you author or obtain the matching `.mobileconfig` file and upload it via Devices → macOS → Configuration → Create → Templates → Custom. This delivers the raw Apple payload Intune's catalog cannot reach.

1. **macOS shell scripts** — a small number of CIS controls are only enforceable by running a

command on the device (for example setting a value with `defaults` or toggling a service). These are delivered via Devices → macOS → Shell scripts, which run as root on a schedule. A macOS **compliance policy** sits alongside these to evaluate and report the device's CIS-relevant posture (for example FileVault on, firewall on, minimum OS version).

CIS Benchmark vs the Microsoft / Apple defaults

Intune and Apple ship their own recommended settings and security defaults. They overlap with CIS but are **not the same**: they reflect Microsoft's or Apple's recommendations, use their own grouping, and revise on their own schedule. CIS is an independent consensus standard with its own control numbering and its own L1/L2 split, and audits ask specifically for CIS. Do not assume a vendor default satisfies a CIS audit — this solution builds the CIS controls explicitly so you can prove the mapping.

The compliance and drift model

Configuring the policy is only half the job. You also:

- **Prove compliance** — Intune reports per-profile and per-device configuration state, so you can

show each Mac received the CIS values.

- **Detect drift** — a device can fall out of compliance (a conflicting profile, a removed profile,

a failed apply, a user change a shell control no longer corrects). The included reporting and validation scripts surface drift so you can reapply.

The benchmark-version maintenance lifecycle (why this solution is two solutions in one)

CIS revises the macOS Benchmark periodically — both for point revisions and, importantly, for each new macOS major release (a new benchmark document ships with each new macOS). Each revision adds, removes, renumbers, or changes the recommended value of controls. Without a process, your hard-won baseline silently falls behind the current benchmark and an audit fails. This solution therefore includes a first-class **Maintenance & Update Guide** (document 4): a repeatable lifecycle of **Obtain** → **Diff** → **Assess/Approve** → **Update** → **Re-validate** → **Record version** that you run every time CIS publishes a new version (or you move Macs to a new macOS major), plus a comparison script that diffs a new CIS export against your live baseline to drive the change.

Source-of-truth note (CIS membership content)

CIS publishes the Benchmark PDF and supporting content (and macOS hardening tooling) behind CIS membership and on CIS WorkBench. The exact CIS control numbers, the `.mobileconfig` payload keys, and the recommended values are the authoritative source — treat them as deferred to the official CIS macOS Benchmark document and CIS WorkBench. This solution gives you the correct procedure to translate the benchmark into the three Intune macOS surfaces; where a specific control is named it is clearly marked as an example only.

Benefits

- **Audit-ready hardening** — every Mac built to the CIS Benchmark, with evidence.
- **Right level, controlled risk** — deliberate L1 baseline with optional, tested L2 hardening.
- **Provable compliance** — per-device configuration state and a fleet posture report.
- **Stays current** — a documented version-update lifecycle so the baseline never silently drifts.
- **Safe rollout** — piloted before fleet-wide, with a rollback note because hardening can break apps and login.

Limitations (be honest with buyers)

- The authoritative control list and values live in the CIS membership content; this solution is the Intune implementation and maintenance method, not a copy of the benchmark.
- Hardening can break apps and even sign-in — L2 especially. A pilot gate and rollback note are mandatory, not optional.
- Some CIS controls are out of scope for Intune (physical, procedural, or hardware items), and some require a custom `.mobileconfig` or a shell script because the settings catalog cannot reach them.
- Achieving 100% of controls may be neither possible nor desirable; documented break-glass exceptions are part of a healthy baseline.

The deployment model

> *Build the Level 1 (and optional Level 2) baseline as Intune settings-catalog policy, custom* `.mobileconfig` *profiles, and macOS shell scripts, assign to a pilot, prove compliance, then expand fleet-wide — and run the maintenance lifecycle on every new benchmark version. The repeatable way:* ***Assess → Configure → Pilot → Validate → Expand.***

2. Requirements Guide — CIS Benchmarks for macOS (Microsoft Intune)

Verify **every** requirement before configuring. Each entry: **Description · Why required · How to verify · How to implement if missing.** Category-wide prerequisites are in `global-requirements-intune-solutions.md`.

Object names used throughout this solution (reuse them verbatim in Intune):

- Settings-catalog policy: `CIS macOS – L1 – Pilot` (and later `CIS macOS – L1 – Production`)
- Optional Level 2 policy: `CIS macOS – L2 – Pilot` / `CIS macOS – L2 – Production`
- Custom `.mobileconfig` profile: `CIS macOS – Custom – Pilot` / `CIS macOS – Custom – Production`
- macOS shell script: `CIS macOS – Shell – Pilot` / `CIS macOS – Shell – Production`
- Compliance policy: `CIS macOS – Compliance`

- Device groups: `CIS macOS – Pilot Devices` and `CIS macOS – Production Devices`
- Version tag (put in every policy/profile/script description): `CIS-macOS-vX.Y.Z` (the benchmark version you are on)

2.1 Intune licensing

Description: An Intune-bearing license (Plan 1 is sufficient for settings catalog, custom profiles, and shell scripts). **Why required:** Intune delivers the settings-catalog policy, custom configuration profiles, macOS shell scripts, and compliance policy. **How to verify:** Microsoft 365 admin center → **Billing** → **Licenses**. **How to implement if missing:** Assign M365 E3/E5 or Business Premium (Intune included), or standalone Intune Plan 1.

2.2 Apple MDM Push (APNs) certificate

Description: A valid Apple Push Notification service (APNs) certificate configured in Intune. **Why required:** Apple devices (including Macs) cannot be managed by Intune without an APNs certificate — no profiles, scripts, or compliance policies reach the Mac until it exists. **How to verify:** Intune → **Devices** → **Enrollment** → **Apple** → **Apple MDM Push certificate**; confirm it is present and **not expired**. **How to implement if missing:** Intune → **Devices** → **Enrollment** → **Apple** → **Apple MDM Push certificate** → **download the CSR**, create the certificate at `https://identity.apple.com` with a long-lived Apple ID, upload it back, and record the renewal date.

2.3 macOS enrollment configured

Description: A working macOS enrollment method so target Macs are enrolled and manageable — Automated Device Enrollment (ADE/DEP), Account-driven (user-enrollment / device enrollment), or Company Portal. **Why required:** Profiles, scripts, and compliance policies apply only to enrolled Macs. **How to verify:** Intune → **Devices** → **macOS** → **macOS devices**; the Mac appears as managed. Run `CISMAC-Discovery.ps1` for tenant/device readiness signals. **How to implement if missing:** Configure **Automated Device Enrollment** (Apple Business/School Manager token + an enrollment profile) for corporate Macs, or **Account-driven**/Company Portal enrollment for user-owned Macs; then enrol the devices.

2.4 Supported macOS version

Description: A currently supported macOS major version that matches a published CIS macOS Benchmark (for example the Sonoma or Sequoia benchmark). **Why required:** Many CIS controls and Apple payloads depend on the macOS major version, and CIS publishes a benchmark per macOS release. Configuring against the wrong version mismatches the controls. **How to verify:** On a Mac, **Apple menu** → **About This Mac**; or Intune → device → **Hardware**. Run `CISMAC-Discovery.ps1`. **How to implement if missing:** Update the Macs to a supported macOS version (and align to the matching CIS benchmark document).

2.5 Official CIS Benchmark source (membership content)

Description: The current CIS Apple macOS Benchmark PDF for your macOS version, and any CIS-provided `.mobileconfig` / audit content from CIS WorkBench. **Why required:** This is the

authoritative list of controls, levels, and values. The exact control numbers, payload keys, and recommended values are deferred to this source — do not work from memory. **How to verify:** You can open the current macOS Benchmark document, identify its version (for example `vX.Y.Z`) and the macOS release it targets, and read the L1/L2 designations. **How to implement if missing:** Obtain CIS membership (or the free benchmark PDF) and download the current macOS Benchmark and any supporting content from CIS WorkBench.

2.6 Custom `.mobileconfig` payloads for non-catalog controls

Description: The `.mobileconfig` files for any CIS controls whose Apple payload is not present in the Intune settings catalog. **Why required:** A minority of CIS controls can only be delivered via a custom configuration profile carrying the raw Apple payload. **How to verify:** While building, note any CIS control with no settings-catalog equivalent; those need a custom `.mobileconfig`. **How to implement if missing:** Author the payload (or obtain CIS-provided content) and upload it via Intune → **Devices** → **macOS** → **Configuration** → **Create** → **Templates** → **Custom**.

2.7 Pilot and production device groups

Description: Two Entra ID security groups: `CIS macOS – Pilot Devices` (a small representative sample of Macs) and `CIS macOS – Production Devices` (the rest of the Mac fleet). **Why required:** Hardening is piloted before fleet rollout because L1/L2 controls can break apps and even login. **How to verify:** Entra → **Groups**; both groups exist. **How to implement if missing:** Entra → **Groups** → **New group** → **Security**; add representative pilot Macs to one and the remaining Macs to the other.

2.8 Change control / approval process

Description: A lightweight, documented approval step for applying and updating the baseline. **Why required:** Hardening and benchmark-version updates are production changes; they need review and a rollback plan. This is central to the maintenance lifecycle in document 4. **How to verify:** You have a place to record approved changes and the benchmark version in use (a change ticket, register, or the policy descriptions). **How to implement if missing:** Adopt a simple change record (who approved, what changed, benchmark version, rollback step). Record the version tag `CIS-macOS-vX.Y.Z` in each policy/profile/script description.

2.9 (Optional) Application inventory for break-glass exceptions

Description: A short list of business-critical apps and workflows that may conflict with hardening (legacy apps, kernel/system extensions, removable-media workflows, AppleScript automation). **Why required:** Lets you plan documented exceptions before they cause an outage or a login failure. **How to verify:** You can name the apps/workflows and which CIS controls might affect them. **How to implement if missing:** Collect the list from app owners during the Assess phase; record planned exceptions in your change record.

Checklist

- Intune licensing assigned

- Apple MDM Push (APNs) certificate present and not expired
- macOS enrollment configured (ADE / Account-driven / Company Portal); Macs enrolled — `CISMAC-Discovery.ps1`
- Supported macOS version matching a published CIS benchmark
- Current CIS macOS Benchmark (and supporting content) obtained — note version `vX.Y.Z`
- Custom `.mobileconfig` payloads for any non-catalog controls
- `CIS macOS – Pilot Devices` and `CIS macOS – Production Devices` groups created
- Change-control / approval process and version tag convention agreed
- (Optional) App inventory for planned break-glass exceptions

Global Requirements — Intune Solutions (category-level)

These requirements apply to **every** Phoenixtekk solution in the **Intune Solutions** category. They are published once at the category level so individual listings stay small. Confirm these before deploying *any* Intune solution. Each entry: **Description · Why · Verify · Implement-if-missing.**

G1. Microsoft Entra ID tenant

Description: An active Microsoft Entra ID (Azure AD) tenant. **Why:** It is the identity backbone for every cloud and Intune-managed resource. **Verify:** Sign in to `https://entra.microsoft.com` as an administrator. **Implement:** Create a Microsoft 365/Azure subscription, which provisions a tenant.

G2. Microsoft Intune subscription

Description: An Intune-bearing license plan. **Why:** Intune delivers all configuration, compliance, and security policies. **Verify:** `https://intune.microsoft.com` loads; **Tenant administration** → **Tenant status** shows active. **Implement:** Assign Microsoft 365 E3/E5, Business Premium, or standalone Intune Plan 1.

G3. Administrative roles

Description: Appropriate Entra/Intune admin roles for the operator. **Why:** Creating policies and enrollment settings requires elevated rights. **Verify:** Entra → **Roles and administrators**; confirm **Intune Administrator**. **Implement:** Assign **Intune Administrator**; reserve Global Administrator for tenant-wide changes.

G4. Device enrollment in Intune

Description: Targeted Windows devices are enrolled and manageable. **Why:** Policies only reach enrolled devices. **Verify:** Intune → **Devices** → **All devices**; the device appears and is **compliant**. **Implement:** Enable **Automatic Enrollment** and join devices to Entra ID.

G5. Pilot security group

Description: A Microsoft Entra security group for pilot/targeted users or devices. **Why:** Every Phoenixtekk solution is piloted before fleet rollout. **Verify:** Entra → **Groups**; the pilot group exists. **Implement:** Entra → **Groups** → **New group** → **Security** → add members.

G6. Supported Windows baseline

Description: Windows 10 21H2+ or Windows 11 with current updates. **Why:** Modern Intune features assume a patched, supported OS. **Verify:** `winver`, or Intune → device → **Hardware**. **Implement:** Deploy **Update rings** or upgrade to Windows 11.

G7. Network access to Microsoft cloud endpoints

Description: Outbound HTTPS to Microsoft 365, Entra, and Intune service URLs. **Why:** Enrollment, policy sync, and authentication depend on these endpoints. **Verify:** From a device, reach `https://login.microsoftonline.com` and `https://enrollment.manage.microsoft.com`. **Implement:** Allow Microsoft's published Office 365/Intune URL & IP ranges through proxy/firewall.

The Phoenixtekk deployment model (all solutions): Assess → Configure → Pilot → Validate → Expand.

3. Implementation Guide (net-new) — CIS Benchmarks for macOS (Microsoft Intune)

Follow these steps **in order**, in the **Microsoft Intune admin center** (`https://intune.microsoft.com`). This builds the CIS Level 1 baseline (and optional Level 2) across the three macOS surfaces — settings catalog, custom `.mobileconfig` profiles, and shell scripts — proves it on a pilot, then expands fleet-wide.

> ***Model:*** *Assess → Configure → Pilot → Validate → Expand. A small* `CIS macOS – Pilot Devices` *group validates the hardening before the* `CIS macOS – Production Devices` *fleet receives it.* ***Hardening can break apps and even sign-in — do not skip the pilot gate or the rollback note (Step 10).***
>
> ***Source note:*** *The authoritative control list, levels, payload keys, and values come from the current CIS macOS Benchmark (Requirement 2.5). Control names shown below are clearly marked examples; configure the actual controls and values from the benchmark you obtained.*

Step 1 — Stage the benchmark (Assess)

1. Open the current CIS macOS Benchmark you obtained and confirm its version (for example `vX.Y.Z`) and the macOS release it targets.
2. Decide scope: **Level 1 for all devices**, and **Level 2 for the high-security subset** (if any).
3. From the benchmark, sort each control into one of the three Intune surfaces: **settings catalog** (most controls, Step 4), **custom** `.mobileconfig` (controls with no catalog payload, Step 6), and **shell script** (controls only enforceable by a command, Step 7). List any planned **break-glass exceptions** (Step 8).

4. Record the version you are targeting as your tag: `CIS-macOS-vX.Y.Z`.

Step 2 — Confirm Apple management prerequisites

1. Intune → **Devices** → **Enrollment** → **Apple** → **Apple MDM Push certificate**; confirm present and not expired (Requirement 2.2).

2. Confirm your target Macs are enrolled: Intune → **Devices** → **macOS** → **macOS devices** (Requirement 2.3).

Step 3 — Create the device groups

1. Entra → **Groups** → **New group** → **Security** → name `CIS macOS – Pilot Devices` → add a few representative Macs. **Create**.

2. Create `CIS macOS – Production Devices` for the rest of the fleet. **Create**.

Step 4 — Build the Level 1 settings-catalog policy

1. Intune → **Devices** → **macOS** → **Configuration** → **Create** → **New Policy** (or **Manage devices** → **Configuration** → **Create**).

2. **Platform:** macOS. **Profile type: Settings catalog**. **Create**.

3. **Name:** `CIS macOS – L1 – Pilot`. **Description:** include the version tag `CIS-macOS-vX.Y.Z`. **Next**.

4. Click **Add settings**. In the settings picker, search for each CIS Level 1 control and add it, then set the value to the benchmark's recommended value. Work through the benchmark sections in order. *Illustrative examples only — use the actual benchmark values:*

• Software Update payload (example: automatic security responses / install macOS updates enabled).

• Security & Privacy / Login Window (example: require password after sleep or screen saver begins).

• Restrictions (example: disable content caching, or restrict AirDrop — per the benchmark).

(Where the benchmark lists a control, confirm it maps to the matching Apple payload in the catalog.)

1. **Next** → **Assignments** → **Add groups** → `CIS macOS – Pilot Devices`. **Next** → **Create**.

Step 5 — (Optional) Build the Level 2 settings-catalog policy

1. Repeat Step 4 with **Name:** `CIS macOS – L2 – Pilot`, adding only the **Level 2** controls (L2 sits on top of L1).

2. **Description:** version tag `CIS-macOS-vX.Y.Z` and "Level 2 — high-security subset only".

3. **Assignments** → **Add groups** → `CIS macOS – Pilot Devices` (only if your pilot includes an L2 Mac). **Create**.

Step 6 — Add custom `.mobileconfig` profiles (only if needed)

1. For any CIS control with no settings-catalog payload (from Step 1.3), obtain or author the matching `.mobileconfig` file (Requirement 2.6).

2. Intune → **Devices** → **macOS** → **Configuration** → **Create** → **New Policy** → **Platform: macOS** → **Profile type: Templates** → **Custom** → **Create**.

3. **Name:** `CIS macOS – Custom – Pilot`. **Description:** version tag. Provide a deployment channel and upload the `.mobileconfig` configuration profile file.

4. **Assignments** → **Add groups** → `CIS macOS – Pilot Devices`. **Create**.

Step 7 — Add macOS shell scripts (only for script-only controls)

1. For any CIS control enforceable only by a command (from Step 1.3): Intune → **Devices** → **macOS** → **Shell scripts** → **Add**.

2. **Name:** `CIS macOS – Shell – Pilot`. **Description:** version tag. Upload the shell script implementing the control (for example a `defaults write` / service-toggle command from the benchmark's remediation guidance).

3. **Script settings:** run as root = **Yes** (most CIS controls require it); set the run frequency so the control is re-enforced on a schedule (re-applying corrects user drift). **Next**.

4. **Assignments** → **Add groups** → `CIS macOS – Pilot Devices`. **Add** → **Next** → **Add**.

Step 8 — Handle break-glass exceptions

1. For each planned exception (a control that would break an app, a workflow, or login), document it in your change record: control, why, compensating control, owner, review date.

2. Implement the exception by **omitting that setting** from the policy/profile/script, or by creating a narrowly scoped exception group and assigning a relaxed policy to it — never silently leave it undocumented.

Step 9 — Create the compliance policy

1. Intune → **Devices** → **macOS** → **Compliance** → **Create policy** → **Platform: macOS** → **Create**.

2. **Name:** `CIS macOS – Compliance`. Set the compliance rules that represent your CIS posture (for example: minimum OS version, **FileVault** required, **firewall** required, system integrity — use the benchmark-relevant rules available in macOS compliance).

3. **Actions for noncompliance:** mark noncompliant after a grace period; optionally email the user.

4. **Assignments** → **Add groups** → `CIS macOS – Pilot Devices`. **Create**.

Step 10 — Apply and validate on the pilot, with pilot gate and rollback note

1. On a pilot Mac, force a sync: **Company Portal** → **...** → **Check status** (or **System Settings** → **Privacy & Security** → **Profiles**, then let it refresh).

2. In Intune, open `CIS macOS – L1 – Pilot` → **Device status / Per-setting status** and confirm **Succeeded**. Check the custom profile and shell script status too (**Devices** → **macOS** → **Shell scripts** → script → **Device status**).

3. On the Mac run `CISMAC-Validation.ps1` (per-device CIS-compliance check) and `CISMAC-Report.ps1` for fleet/pilot posture and drift.

4. **Soak the pilot:** have pilot users run their real apps for several business days. Watch for breakage (sign-in/login window, app launch under Gatekeeper, firewall-blocked apps, printing, removable media, automation). Complete the **Testing Guide** (document 5).

5. **Gate:** do not expand until the pilot passes the Testing Guide gate AND no unresolved app/login breakage remains.

6. **Rollback (if hardening breaks something):** in Intune, **remove the assignment** of the offending profile/policy/script from the affected group (or set the specific setting to *Not configured*, or disable the shell script), then sync. Removing the assignment is the fastest safe rollback; the Mac reverts on the next policy cycle. Record the rollback and the offending control as a break-glass exception (Step 8).

Step 11 — Expand to production

1. Duplicate each piloted policy (Intune → policy → **Duplicate** where available) and rename the copies `CIS macOS – L1 – Production`, `CIS macOS – L2 – Production`, `CIS macOS – Custom – Production`, `CIS macOS – Shell – Production`. Keep the version tag in the description. (For shell scripts, re-create with the production name and assignment.)

2. Reassign the production copies to `CIS macOS – Production Devices` (or simply add that group to the existing policies if you prefer a single policy set). Assign `CIS macOS – Compliance` to production too.

3. Roll out in waves if the fleet is large; re-run `CISMAC-Report.ps1` after each wave to confirm posture and catch drift.

Order of operations

`Stage benchmark → Confirm APNs/enrollment → Groups → L1 catalog policy → (L2 catalog) → Custom .mobileconfig → Shell scripts → Exceptions → Compliance → Apply + Validate (pilot gate / rollback) → Expand`

4. Maintenance & Update Guide — the CIS benchmark-version lifecycle (macOS)

This is the half of the solution that keeps your baseline from silently drifting behind CIS. CIS revises the macOS Benchmark periodically, and — importantly for macOS — publishes a **new benchmark document for each new macOS major release**. Each revision can add, remove, renumber, or change the recommended value of controls, and a new macOS release can change which Apple payloads exist. Run this lifecycle **every time CIS publishes a new version (or you move Macs to a new macOS major)**, and run the drift portion **on an ongoing cadence** even between versions.

> ***Maintenance model (mirrors the deployment model): Obtain → Diff → Assess/Approve → Update (controlled change) → Re-validate → Record version.*** *Treat every benchmark update as a piloted, approved, rollback-ready production change — exactly like the net-new implementation in document 3.*

Two kinds of maintenance

1. **Ongoing drift maintenance (continuous):** devices fall out of compliance between benchmark versions (a conflicting profile, a removed profile, a failed apply, or a user change that a shell control has not yet re-corrected). You detect and reapply.

1. **Benchmark-version update (event-driven):** CIS ships a new version, or you adopt a new macOS major; you bring your baseline up to it. This is the main subject of this guide.

Part A — Ongoing drift maintenance (run on a cadence)

1. On a schedule (for example weekly), run `CISMAC-Report.ps1` to produce the fleet CIS posture and a drift list (Macs whose configuration/script state is no longer compliant).

1. For drifted devices, run `CISMAC-Validation.ps1` to see which settings drifted on a specific Mac.

2. Reapply with `CISMAC-Remediation.ps1`. **It runs with `-WhatIf` by default** — review the planned actions first, then re-run with `-WhatIf:$false` to repair/reassign the drifted policy/profile/ script so the baseline re-applies. (Remediation here means making the Intune-managed objects re-apply — re-syncing the device and confirming assignments — not editing settings on the Mac outside Intune.)

1. If a Mac keeps drifting, look for a **conflicting profile** in Intune (two profiles setting the same payload differently) and resolve the conflict at the policy level. For a control delivered by a shell script that users keep changing, confirm the script's **run frequency** re-enforces it often enough.

Part B — Benchmark-version update (run when CIS releases a new version)

Step 1 — Obtain

1. Download the **new** CIS macOS Benchmark version (and any updated supporting `.mobileconfig` / audit content) from CIS WorkBench. If this is a new macOS major, obtain that release's benchmark document.

2. Note the new version number (for example `vX.Y.Z+1`) and keep the version you are currently live on (your existing tag `CIS-macOS-vX.Y.Z`).

Step 2 — Diff (what changed)

1. Export your **current live baseline** from Intune (the settings in `CIS macOS – L1 – Production`, any L2 catalog policy, the custom `.mobileconfig` payloads, and the shell-script controls). The discovery/report scripts and the Intune export help here.

2. Export or extract the **new benchmark's** intended settings (from the new benchmark / supporting content).

3. Run `CISMAC-Compare-BenchmarkVersion.ps1` to diff the new benchmark export against your current live baseline. It reports:

• **Added** controls (new in this version — you must add them),

• **Removed** controls (dropped — you may retire them),

- **Changed** controls (same control, new recommended value — you must update the value),
- **Level changes** (a control moved between L1 and L2),
- **Surface changes** (where indicated — a control now needs a different macOS surface, for example moved from a custom profile into the settings catalog).

1. Save the diff output as the change record for this version update.

Step 3 — Assess / Approve

1. For each changed/added control, assess **business impact** — could it break an app, a workflow, or login? Pay special attention to controls moving into L1 (now applies fleet-wide) and to Gatekeeper, firewall, removable-media, automation, and authentication controls.
1. Decide for each: **adopt**, **adopt with an exception**, or **defer with documented reason**.
2. Obtain approval per your change-control process (Requirement 2.8). Record approver, date, and the list of accepted changes.

Step 4 — Update (controlled change, piloted)

1. **Update the pilot objects first.** Edit `CIS macOS – L1 – Pilot` (and the L2 catalog policy, `CIS macOS – Custom – Pilot`, and `CIS macOS – Shell – Pilot`) to add/change/remove the approved controls. For a control that changed surface, move it to the correct surface. Update each object **Description** to the new version tag `CIS-macOS-vX.Y.Z+1`.
1. Implement approved exceptions exactly as in document 3, Step 8 (omit the setting, scope an exception group, or disable the script); keep the exception register current.
1. Assignment remains `CIS macOS – Pilot Devices`. Sync a pilot Mac.

Step 5 — Re-validate (pilot gate again)

1. In Intune, confirm the updated pilot objects show **Succeeded** (per-setting and shell-script status).
2. Run `CISMAC-Validation.ps1` on a pilot Mac and `CISMAC-Report.ps1` for pilot posture.
3. **Soak the pilot** for several business days against real apps — a new benchmark version (or a new macOS major) can introduce a control that breaks something the previous baseline never touched. Complete the relevant Testing Guide checks.
1. **Gate:** do not promote to production until the pilot passes and no unresolved breakage remains.

If something breaks, roll back that control (remove the setting / unassign / disable the script) and record it as an exception, exactly as in document 3, Step 10.

Step 6 — Promote and Record version

1. Apply the same approved changes to the production objects (`CIS macOS – L1 – Production`, and the L2/custom/shell production objects as applicable). Update their **Description** to the new version tag. Roll out in waves for large fleets.
1. Re-run `CISMAC-Report.ps1` after each wave to confirm posture and catch drift.

2. **Record the version** you are now live on: update the change record, the policy/profile/script descriptions (`CIS-macOS-vX.Y.Z+1`), and your baseline document. This closes the loop — the next benchmark release diffs against this newly recorded baseline.

Maintenance order of operations

```
Obtain new version → Diff vs live baseline (CISMAC-Compare-BenchmarkVersion.ps1) → Assess/Approve → Update pilot objects (controlled change) → Re-validate (pilot gate + rollback) → Promote to production → Record new version tag
```

Cadence summary

Activity	When	Tool
Drift detection + reapply	Weekly (or your cadence)	`CISMAC-Report.ps1`, `CISMAC-Validation.ps1`, `CISMAC-Remediation.ps1`
Benchmark-version update	Each new CIS macOS Benchmark release (and each new macOS major)	`CISMAC-Compare-BenchmarkVersion.ps1` + this Part B
APNs certificate renewal	Annually, before expiry	Intune → Devices → Enrollment → Apple → Apple MDM Push certificate
Version recorded in descriptions	After every update	Intune object Description = `CIS-macOS-vX.Y.Z`

5. Testing Guide — CIS Benchmarks for macOS (Microsoft Intune)

Run after the pilot objects apply (Implementation Step 10) and again after each benchmark-version update (Maintenance Part B, Step 5).

Test 1 — Settings-catalog policy applied to the pilot

Do: Intune → **Devices** → **macOS** → **Configuration** → `CIS macOS – L1 – Pilot` → **Device status** (and **Per-setting status**). **Expected:** Pilot Macs show **Succeeded**; per-setting status shows the CIS settings applied without conflicts.

Test 2 — Settings present on the device

Do: On a pilot Mac run `CISMAC-Validation.ps1`. **Expected:** The script reports the Mac's CIS-relevant settings match the expected baseline values; no drift flagged.

Test 3 — Compliance policy evaluates the device

Do: Intune → **Devices** → **macOS** → **Compliance** → `CIS macOS – Compliance` → **Device status**; or on the Mac check **Company Portal**. **Expected:** Pilot Macs evaluate **Compliant** against the CIS-relevant compliance rules (for example FileVault on, firewall on, minimum OS version).

Test 4 — Custom `.mobileconfig` profile applied (if used)

Do: Intune → `CIS macOS – Custom – Pilot` → **Device status**; on the Mac confirm via **System Settings** → **Privacy & Security** → **Profiles**. **Expected:** The custom profile shows **Succeeded** and the payload is present on the Mac.

Test 5 — Shell-script control applied (if used)

Do: Intune → **Devices** → **macOS** → **Shell scripts** → `CIS macOS – Shell – Pilot` → **Device status**; confirm the resulting setting on the Mac. **Expected:** The script shows it ran successfully and the targeted control value is set.

Test 6 — Level 2 controls present (if deployed)

Do: On an L2 pilot Mac, run `CISMAC-Validation.ps1` (it includes L2 checks when the L2 policy is assigned). **Expected:** L2 settings are present in addition to L1; no L1/L2 conflicts.

Test 7 — App and login soak (hardening did not break the business)

Do: Pilot users run their real apps for several business days (sign-in / login window, app launch under Gatekeeper, firewall-blocked apps, printing, removable media, automation). **Expected:** No business-app or login breakage. Any breakage is traced to a control and handled as a break-glass exception or rolled back.

Test 8 — Fleet posture and drift report

Do: Run `CISMAC-Report.ps1`. **Expected:** A clear per-device CIS posture (compliant / drifted) and a drift list; pilot Macs appear compliant.

Test 9 — Benchmark-version diff (maintenance dry-run)

Do: Run `CISMAC-Compare-BenchmarkVersion.ps1` against the current live baseline using your current benchmark export. **Expected:** With no new version, the diff is empty (added/removed/changed = 0) — proving the script and your baseline agree before a real version update.

Pass/fail gate

Gate	Requirement
Pilot pass	Tests 1–7 pass; no unresolved business-app or login breakage on the pilot
Production pass	Tests 1–3 and 8 pass for `CIS macOS – Production Devices` after each rollout wave

Gate	Requirement
Version-update pass	After a benchmark update: Tests 1–7 re-pass on the pilot before promoting to production

6. Troubleshooting Guide — CIS Benchmarks for macOS (Microsoft Intune)

Use `CISMAC-Discovery.ps1` / `CISMAC-Validation.ps1` for tenant/device state, `CISMAC-Report.ps1` for fleet posture and drift, `CISMAC-Remediation.ps1` for safe reapply (default `-WhatIf`), and `CISMAC-Compare-BenchmarkVersion.ps1` when working a version update.

A. Policy/profile not applying / device not compliant

Symptom	Cause	Remediation
Settings-catalog policy shows Error on a Mac	Mac not enrolled / not synced / not in the assigned group	Confirm enrollment (`CISMAC-Discovery.ps1`) and group membership; force a sync (Company Portal → Check status); re-check **Per-setting status**
Nothing applies to any Mac	Apple MDM Push (APNs) certificate missing or expired	Intune → **Devices** → **Enrollment** → **Apple** → **Apple MDM Push certificate**; renew with the same Apple ID (Requirement 2.2)
Some settings Succeeded, others Error	A control's Apple payload is unavailable on that macOS version	Confirm the macOS version matches the CIS benchmark; deliver the control via custom `.mobileconfig` or shell script instead
Mac drifts back after applying	A conflicting Intune profile sets the same payload differently	Find the conflict (`CISMAC-Report.ps1` + Intune per-setting status) and resolve it at the profile level, not by repeated remediation

B. Hardening broke an app or login

• Identify the offending control from the app/login behavior and `CISMAC-Validation.ps1`.

• **Fastest safe rollback:** remove the profile/policy/script assignment from the affected group, or set that single control to **Not configured** (or disable the shell script), then sync; the Mac reverts on the next policy cycle.

• Record the rollback as a documented break-glass exception (control, reason, compensating control, owner, review date).

C. Custom `.mobileconfig` profile not taking effect

Symptom	Cause	Remediation
Custom profile assigned but payload absent	Wrong deployment channel, malformed `.mobileconfig`, or a required user approval not granted	Re-upload a valid `.mobileconfig`; confirm the deployment channel; on the Mac check **System Settings** → **Privacy & Security** → **Profiles**
Catalog setting and custom profile fight	The same payload set in two places	Keep each control in one surface only — prefer the settings catalog, remove the duplicate from the custom profile

D. macOS shell-script control not enforcing

Symptom	Cause	Remediation
Script shows failed in Device status	Script error, or not run as root	Open **Devices** → **macOS** → **Shell scripts** → script → review the error; set **Run script as signed-in user = No** (run as root); fix the command
Control reverts after a user change	Run frequency too infrequent to re-enforce	Increase the shell script's run frequency so it re-applies the control on a schedule

E. Compliance policy reports noncompliant unexpectedly

- Confirm the compliance rules match controls the Mac can actually meet (minimum OS version, FileVault, firewall).
- Allow the grace period to elapse and let the Mac re-evaluate; sync if needed.
- Cross-check with `CISMAC-Validation.ps1` to confirm whether it is a real gap or a reporting lag.

F. Benchmark-version update issues

Symptom	Cause	Remediation
`CISMAC-Compare-BenchmarkVersion.ps1` shows huge unexpected diff	Comparing against the wrong export, or baseline not exported correctly	Re-export the current live baseline and the new benchmark; confirm you are diffing the right two files
New L1 control breaks Macs fleet-wide after promotion	Promoted to production without a full pilot soak	Roll back that control (unassign / Not configured / disable script); re-pilot per Maintenance Part B before promoting again

Symptom	Cause	Remediation
Controls missing after a macOS major upgrade	The benchmark (and some Apple payloads) changed for the new macOS release	Obtain the new macOS release's benchmark and run the full Maintenance Part B lifecycle

G. Graph scripts fail to connect

- `CISMAC-Report.ps1` needs Microsoft Graph PowerShell and `DeviceManagementConfiguration.Read.All`; consent the scope on first `Connect-MgGraph`.
- If module load fails, run `Install-Module Microsoft.Graph -Scope CurrentUser`.

H. Diagnostics quick reference

```
Get-MgContext   # confirm Graph scopes
```

On the Mac (Terminal, for local checks of CIS-relevant state):

```
sw_vers                                 # macOS product version / build
fdesetup status                         # FileVault on/off
profiles list                           # installed configuration profiles
```

Intune-side: device → **Device configuration** shows each profile's apply status; policy → **Per-setting status** shows which CIS settings succeeded or errored; **Shell scripts** → **Device status** shows script results.

I. When to escalate

If the profile shows Succeeded, the Mac is enrolled, synced, and in the right group, the APNs certificate is valid, no conflicting profile exists, and a control still will not apply, capture the per-setting status, the shell-script result, and the on-device `profiles list` output and open a Microsoft support case.

7. Best Practices — CIS Benchmarks for macOS (Microsoft Intune)

- **Pilot before you harden the fleet.** A small `CIS macOS – Pilot Devices` group soaked against real apps catches breakage before it reaches production — the single most important practice, because hardening can break apps and even login.
- **Start at Level 1, add Level 2 deliberately.** Deploy L1 broadly; apply L2 only to the high-security subset that needs it and only after extra testing. Never enable L2 fleet-wide on day one.
- **Work from the official benchmark, not memory.** The authoritative control list, levels, payload keys, and values live in the current CIS macOS Benchmark for your macOS version. Always configure from the version you obtained.
- **Use the right macOS surface for each control.** Prefer the **settings catalog** first; use a **custom** `.mobileconfig` only for controls the catalog cannot reach; use a **shell script** only for controls enforceable solely by command — and set its run frequency so it re-enforces against user drift.

• **Keep the APNs certificate alive.** Apple management stops the moment the Apple MDM Push certificate expires. Track its renewal date and renew with the same Apple ID before expiry.

• **Name and tag everything consistently.** Use the `<Purpose> – <Scope> – <Stage>` convention (`CIS macOS – L1 – Pilot`, etc.) and put the benchmark version tag `CIS-macOS-vX.Y.Z` in every policy/profile/script description so the live version is always visible and auditable.

• **Document every break-glass exception.** A healthy baseline has documented exceptions, not silent gaps — record the control, reason, compensating control, owner, and review date.

• **Treat each new benchmark version (and each new macOS major) as a change, not a footnote.** Run the full Obtain → Diff → Assess/Approve → Update → Re-validate → Record lifecycle. Diff with `CISMAC-Compare-BenchmarkVersion.ps1` so you never miss an added or changed control.

• **Detect drift on a cadence.** Run `CISMAC-Report.ps1` regularly and reapply with `CISMAC-Remediation.ps1` (default `-WhatIf`) so Macs don't quietly fall out of compliance between versions.

• **Resolve policy conflicts at the source.** If a Mac repeatedly drifts, find the conflicting Intune profile setting the same payload differently — don't just keep remediating the symptom.

• **Keep a rollback ready.** Removing a profile/script assignment (or setting a control to *Not configured*) is the fastest safe rollback when hardening breaks an app or login; record what you rolled back as an exception.

• **Prove compliance with evidence.** Use Intune per-setting/device status plus the report script as your audit evidence that each Mac received the CIS values.

8. Deliverables Summary — CIS Benchmarks for macOS (Microsoft Intune)

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	What CIS Benchmarks are, L1 vs L2, how controls map to the three Intune macOS surfaces (settings catalog + custom `.mobileconfig` + shell scripts), CIS vs vendor defaults, compliance/drift model, version lifecycle
2. Requirements Guide	Every prerequisite with Why / Verify / Implement-if-missing (incl. APNs and macOS enrollment), plus the object-naming convention
Global Requirements (Intune Solutions)	Category-wide prerequisites, published once
3. Implementation Guide (net-new)	Exact, ordered Intune steps: stage benchmark, build L1/L2 catalog, add custom profiles and shell scripts, exceptions, compliance, pilot, validate, rollback, expand

Document	Purpose
4. Maintenance & Update Guide	The CIS benchmark-version lifecycle: Obtain → Diff → Assess/Approve → Update → Re-validate → Record, plus ongoing drift maintenance
5. Testing Guide	Validation procedures with pass/fail gates (including a maintenance dry-run)
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Field-proven recommendations

PowerShell scripts

Script	What it does
`CISMAC-Discovery.ps1`	Tenant/device CIS-readiness for macOS: APNs presence, enrolled Macs, macOS version spread, and conflicting/duplicate macOS policy authority (read-only)
`CISMAC-Validation.ps1`	Per-device CIS-compliance check against the expected baseline values, by reading Intune device config states for the Mac (read-only)
`CISMAC-Report.ps1`	Fleet CIS posture + drift via Microsoft Graph (macOS configuration policies, custom profiles, and shell-script run states) (read-only)
`CISMAC-Remediation.ps1`	Reapplies/repairs drifted policy/profile/script assignments by re-syncing and confirming assignment; `-WhatIf` by default
`CISMAC-Compare-BenchmarkVersion.ps1`	Diffs a NEW CIS benchmark export against the current live baseline to drive the update process

Object naming convention (used across docs and scripts)

- Settings-catalog policies: `CIS macOS – L1 – Pilot` / `CIS macOS – L1 – Production` (and `– L2 –` variants)
- Custom `.mobileconfig` profile: `CIS macOS – Custom – Pilot` / `CIS macOS – Custom – Production`
- macOS shell script: `CIS macOS – Shell – Pilot` / `CIS macOS – Shell – Production`
- Compliance policy: `CIS macOS – Compliance`
- Device groups: `CIS macOS – Pilot Devices` / `CIS macOS – Production Devices`
- Version tag in every policy/profile/script description: `CIS-macOS-vX.Y.Z`

What the buyer receives

- These guides as a branded **Word** document and **PDF**.
- All five **PowerShell scripts** in a single ZIP.
- A repeatable process for both net-new implementation and ongoing version maintenance:

Assess → **Configure** → **Pilot** → **Validate** → **Expand** (and **Obtain** → **Diff** → **Assess/Approve** → **Update** → **Re-validate** → **Record** for each new benchmark version).

Outcome

Every Mac hardened to the CIS Benchmark and provably compliant — with a documented update lifecycle that keeps the baseline current each time CIS revises it (or you move to a new macOS major), proven on a pilot before you scale.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

CISMAC-Compare-BenchmarkVersion.ps1

Purpose: Drives the benchmark-version update lifecycle (Maintenance Guide, Part B, Step 2). Compares a NEW CIS macOS export (from the new benchmark / supporting content) against your CURRENT live baseline export, and reports which controls were ADDED, REMOVED, CHANGED (new recommended value), moved LEVEL (L1<->L2), or moved SURFACE (catalog / custom profile / shell script). Pure file comparison — read-only, makes no changes and does not touch the tenant. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\CISMAC-Compare-BenchmarkVersion.ps1 ` -CurrentPath .\baseline-current.json -NewPath .\benchmark-new.json -CsvPath C:\Temp\cismac-diff.csv Each JSON is an array of controls, e.g.: [{ "Id":"2.3.1", "Name":"...", "Level":"L1", "Surface":"catalog", "Value":"true" }] Export the CURRENT file from your live Intune baseline; export the NEW file from the new CIS macOS benchmark. (Control IDs/values/surfaces are deferred to the official CIS Benchmark.) EXPECTED RESULTS Counts and lists of Added / Removed / Changed / Level-changed / Surface-changed controls, and an optional CSV change record to attach to your approval ticket. Exit code 0 = identical, 1 = differences found. .NOTES Intune Solutions — CIS macOS. Read-only. Use the output to assess/approve the version update. #> [CmdletBinding()] param([Parameter(Mandatory=$true)][string]$CurrentPath, [Parameter(Mandatory=$true)][string]$NewPath, [string]$CsvPath)

How to run: powershell -ExecutionPolicy Bypass -File .\CISMAC-Compare-BenchmarkVersion.ps1 ` -CurrentPath .\baseline-current.json -NewPath .\benchmark-new.json -CsvPath C:\Temp\cismac-diff.csv Each JSON is an array of controls, e.g.: [{ "Id":"2.3.1", "Name":"...", "Level":"L1", "Surface":"catalog", "Value":"true" }] Export the CURRENT file from your live Intune baseline; export the NEW file from the new CIS macOS benchmark. (Control IDs/values/surfaces are deferred to the official CIS Benchmark.) EXPECTED RESULTS Counts and lists of Added / Removed / Changed / Level-changed / Surface-changed controls, and an optional CSV change record to attach to your approval ticket. Exit code 0 = identical, 1 = differences found. .NOTES Intune Solutions — CIS macOS. Read-only. Use the output to assess/approve the version update. #> [CmdletBinding()] param([Parameter(Mandatory=$true)][string]$CurrentPath, [Parameter(Mandatory=$true)][string]$NewPath, [string]$CsvPath)

CISMAC-Discovery.ps1

Purpose: Reports whether the tenant and its Macs are ready to receive a CIS macOS baseline: that an Apple MDM Push (APNs) certificate exists and is not near expiry, how many managed Macs are enrolled, the macOS version spread across the fleet, and whether any existing macOS configuration policies already target the same area (potential conflicts). Read-only against Microsoft Graph (Intune). HOW TO RUN powershell -ExecutionPolicy Bypass -File .\CISMAC-Discovery.ps1 Requires the Microsoft Graph SDK and these read scopes (consented on first sign-in): DeviceManagementServiceConfig.Read.All, DeviceManagementConfiguration.Read.All, DeviceManagementManagedDevices.Read.All. EXPECTED RESULTS APNs certificate presence/expiry, count of enrolled Macs, a macOS-version breakdown, and a list of existing macOS configuration policies to review for conflicts before you build. .NOTES Intune Solutions — CIS macOS. Read-only against Graph. CIS control values are deferred to the benchmark. #> [CmdletBinding()] param()

How to run: powershell -ExecutionPolicy Bypass -File .\CISMAC-Discovery.ps1 Requires the Microsoft Graph SDK and these read scopes (consented on first sign-in): DeviceManagementServiceConfig.Read.All, DeviceManagementConfiguration.Read.All, DeviceManagementManagedDevices.Read.All. EXPECTED RESULTS APNs certificate presence/expiry, count of enrolled Macs, a macOS-version breakdown, and a list of existing macOS configuration policies to review for conflicts before you build. .NOTES Intune Solutions — CIS macOS. Read-only against Graph. CIS control values are deferred to the benchmark. #> [CmdletBinding()] param()

CISMAC-Remediation.ps1

Purpose: When CISMAC-Validation / CISMAC-Report shows drift, this makes the Intune-managed CIS macOS objects re-apply to the affected Macs: it issues a device sync action (so the configuration policies, custom profiles, and shell scripts re-evaluate) and verifies that each drifted Mac is still in the assigned CIS macOS group. It does NOT hand-edit settings on the Mac — Intune remains the single source of truth; on-device enforcement stays in the Apple .mobileconfig/shell controls. HOW TO RUN Preview (default): powershell -ExecutionPolicy Bypass -File .\CISMAC-Remediation.ps1 -PolicyNameLike "CIS macOS" Apply: powershell -ExecutionPolicy Bypass -File .\CISMAC-Remediation.ps1 -PolicyNameLike "CIS macOS" -WhatIf:$false Requires the Microsoft Graph SDK and DeviceManagementConfiguration.Read.All + DeviceManagementManagedDevices.PrivilegedOperations.All (for the sync action). EXPECTED RESULTS With -WhatIf (default): lists the drifted Macs and the sync actions it WOULD issue. With -WhatIf:$false: issues a sync to each drifted Mac so the CIS macOS objects re-apply. Re-run CISMAC-Validation.ps1 after a policy cycle. .NOTES Intune Solutions — CIS macOS. Defaults to -WhatIf (no change unless -WhatIf:$false). #> [CmdletBinding(SupportsShouldProcess=$true, ConfirmImpact='High')] param([string]$PolicyNameLike = 'CIS macOS')

How to run: Preview (default): powershell -ExecutionPolicy Bypass -File .\CISMAC-Remediation.ps1 -PolicyNameLike "CIS macOS" Apply: powershell -ExecutionPolicy Bypass -File .\CISMAC-Remediation.ps1 -PolicyNameLike "CIS macOS" -WhatIf:$false Requires the Microsoft Graph SDK and DeviceManagementConfiguration.Read.All + DeviceManagementManagedDevices.PrivilegedOperations.All (for the sync action). EXPECTED RESULTS With -WhatIf (default): lists the drifted Macs and the sync actions it WOULD issue. With -WhatIf:$false: issues a sync to each drifted Mac so the CIS macOS objects re-apply. Re-run CISMAC-Validation.ps1 after a policy cycle. .NOTES Intune Solutions — CIS macOS. Defaults to -WhatIf (no change unless -WhatIf:$false). #> [CmdletBinding(SupportsShouldProcess=$true, ConfirmImpact='High')] param([string]$PolicyNameLike = 'CIS macOS')

CISMAC-Report.ps1

Purpose: Reports, across the Mac fleet, how each device evaluates against the CIS macOS objects in Intune — by reading the device statuses for the named settings-catalog policies and custom configuration profiles, plus the run states of the CIS macOS shell scripts. Summarizes compliant vs drifted (error/conflict/failed/notApplicable) per object. Read-only against Graph. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\CISMAC-Report.ps1 -PolicyNameLike "CIS macOS" -CsvPath C:\Temp\cismac-posture.csv Requires the Microsoft Graph SDK and DeviceManagementConfiguration.Read.All. EXPECTED RESULTS A per-object breakdown of device states (succeeded/error/conflict/failed/etc.) and an optional per-device CSV showing which Macs have drifted. .NOTES Intune Solutions — CIS macOS. Read-only against Graph. #> [CmdletBinding()] param([string]$PolicyNameLike = 'CIS macOS', [string]$CsvPath)

How to run: powershell -ExecutionPolicy Bypass -File .\CISMAC-Report.ps1 -PolicyNameLike "CIS macOS" -CsvPath C:\Temp\cismac-posture.csv Requires the Microsoft Graph SDK and DeviceManagementConfiguration.Read.All. EXPECTED RESULTS A per-object breakdown of device states (succeeded/error/conflict/failed/etc.) and an optional per-device CSV showing which Macs have drifted. .NOTES Intune Solutions — CIS macOS. Read-only against Graph. #> [CmdletBinding()] param([string]$PolicyNameLike = 'CIS macOS', [string]$CsvPath)

CISMAC-Validation.ps1

Purpose: Checks one Mac's effective Intune configuration state against the CIS macOS baseline policies it should have, and reports per-policy PASS/DRIFT. It reads, for the named device, the Intune deviceConfigurationStates / per-policy status for the CIS macOS objects (settings catalog, custom profiles, shell scripts) and flags any that are not 'compliant'/'succeeded'. Read-only — it does not change settings. (On-device CIS enforcement is delivered by Apple .mobileconfig payloads and shell scripts; this script reports the Intune view of them.) HOW TO RUN powershell -ExecutionPolicy Bypass -File .\CISMAC-Validation.ps1 -DeviceName "MAC-001" -PolicyNameLike "CIS macOS" Requires the Microsoft Graph SDK and DeviceManagementManagedDevices.Read.All + DeviceManagementConfiguration.Read.All. EXPECTED RESULTS A per-policy PASS/DRIFT table for that Mac and "DEVICE COMPLIANT" when no drift is found. .NOTES Intune Solutions — CIS macOS. Read-only. Control values are deferred to the CIS Benchmark. #> [CmdletBinding()] param([Parameter(Mandatory=$true)][string]$DeviceName, [string]$PolicyNameLike = 'CIS macOS')

How to run: powershell -ExecutionPolicy Bypass -File .\CISMAC-Validation.ps1 -DeviceName "MAC-001" -PolicyNameLike "CIS macOS" Requires the Microsoft Graph SDK and DeviceManagementManagedDevices.Read.All + DeviceManagementConfiguration.Read.All. EXPECTED RESULTS A per-policy PASS/DRIFT table for that Mac and "DEVICE COMPLIANT" when no drift is found. .NOTES Intune Solutions — CIS macOS. Read-only. Control values are deferred to the CIS Benchmark. #> [CmdletBinding()] param([Parameter(Mandatory=$true)][string]$DeviceName, [string]$PolicyNameLike = 'CIS macOS')